

UJIAN AKHIR SEMESTER
MATA KULIAH KECERDASAN BUATAN
Klasifikasi Popularitas Film Menggunakan Algoritma Decision
Tree (Studi Kasus: Data TMDb)



Nama:

Erdhien Nizar Tharegi 23.01.5076

Azrilla Dimas Putra Tama 23.01.5078

Muhammad Rizki Fajri 23.01.5094

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS AMIKOM YOGYAKARTA

2025

Pendahuluan

Industri film modern menghasilkan volume data yang sangat besar, yang dapat dimanfaatkan untuk memahami tren pasar dan preferensi audiens. Proyek ini bertujuan untuk menerapkan salah satu metode dasar Machine Learning untuk menyelesaikan sebuah kasus klasifikasi menggunakan data film dari The Movie Database (TMDb).

Studi kasus ini berfokus pada pembangunan model klasifikasi untuk memprediksi apakah sebuah film termasuk dalam kategori "Sangat Populer" atau "Tidak Populer". Dengan menggunakan algoritma Decision Tree (Pohon Keputusan), model akan dilatih untuk mengenali pola dari fitur-fitur seperti tahun rilis dan rata-rata rating film. Tujuan akhirnya adalah untuk menunjukkan bagaimana algoritma klasifikasi dapat digunakan untuk membuat prediksi berdasarkan data numerik dan bagaimana kinerjanya dapat dievaluasi secara visual.

Deskripsi Dataset

- Sumber Data: Data primer proyek ini berasal dari API publik The Movie Database (TMDb), yang sebelumnya telah di-crawl dan disimpan dalam file `tmdb_movies.csv`.
- Jumlah Data: Dataset berisi informasi dari 100 film terpopuler pada saat pengambilan data.
- Fitur yang Digunakan:
 - `vote_average` (Fitur 1): Rata-rata rating yang diberikan oleh pengguna (numerik).
 - `year` (Fitur 2): Tahun rilis film (numerik).
 - `popularity` (Untuk Target): Skor popularitas yang digunakan untuk membuat label kelas.

- Label Kelas (Target): Sebuah kolom baru bernama `is_highly_popular` dibuat sebagai target prediksi. Sebuah film diberi label 1 (Sangat Populer) jika skor popularity-nya di atas 500, dan 0 (Tidak Populer) jika sebaliknya.

Penjelasan Algoritma

3.1 Konsep Dasar

Decision Tree (Pohon Keputusan) adalah salah satu algoritma supervised learning yang paling populer dan intuitif. Algoritma ini bekerja dengan cara membangun model prediksi dalam bentuk struktur pohon. Prosesnya dimulai dari satu node akar (sampel data utuh) dan secara berulang memecah data menjadi sub-kelompok yang lebih kecil berdasarkan aturan keputusan sederhana pada fitur-fitur data.

Setiap node internal pada pohon merepresentasikan sebuah "tes" pada suatu fitur (misalnya, "Apakah `vote_average` ≤ 7.5 ?"), dan setiap cabang merepresentasikan hasil dari tes tersebut (Ya atau Tidak). Proses pemecahan ini berlanjut hingga mencapai node daun (leaf node), di mana pada titik tersebut keputusan akhir (label kelas) dibuat.

3.2 Kecocokan Algoritma untuk Kasus Terpilih

Decision Tree sangat cocok untuk kasus klasifikasi popularitas film ini karena beberapa alasan yaitu, hasil dari model Decision Tree dapat divisualisasikan dalam bentuk pohon yang alur logikanya sangat mudah dipahami oleh manusia. Kita bisa melihat dengan jelas aturan-aturan apa yang digunakan model untuk membuat keputusan. Algoritma ini merupakan fondasi penting dalam machine learning dan sangat ideal untuk tugas klasifikasi sederhana seperti ini. Penanganan Data Numerik dengan Decision Tree dapat secara efisien menangani fitur-fitur numerik dengan menemukan nilai ambang batas terbaik untuk memecah data.

3.3 Penerapan Konsep Probabilitas

Konsep probabilitas diterapkan dalam Decision Tree melalui metrik yang digunakan untuk mengukur kualitas sebuah pemecahan data, yaitu Gini Impurity. Gini Impurity adalah ukuran probabilitas sebuah elemen yang dipilih secara acak akan salah diklasifikasikan. Nilainya berkisar dari 0 hingga 0.5, di mana 0 berarti semua elemen di dalam node memiliki kelas yang sama (murni), dan 0.5 berarti kelas terdistribusi secara acak (50/50). Pada setiap langkah, algoritma Decision Tree akan memilih fitur dan ambang batas yang menghasilkan penurunan Gini Impurity terbesar. Artinya, model secara probabilistik mencari aturan yang paling efektif untuk mengurangi ketidakpastian dan memurnikan kelompok data, sehingga prediksi menjadi lebih akurat.

Perhitungan Manual dengan 3 Baris Data

Untuk melakukan perhitungan pada project ini adalah dengan menggunakan model Pohon Keputusan yang telah dilatih yaitu, Model Pohon Keputusan (Decision Tree). Untuk melakukan simulasi perhitungan manual akan menguji 3 film dan menelusuri jalurnya di dalam pohon keputusan untuk mendapatkan prediksi.

Data yang Akan Diuji:

title	vote_average	year
How to Train Your Dragon 2	7.677	2014
M3GAN 2.0	7.735	2025
xXx	5.945	2002

1. Perhitungan untuk "How to Train Your Dragon 2"

- Fitur: $\text{vote_average} = 7.677$, $\text{year} = 2014$.
- Langkah 1 (Node Akar): Apakah $\text{vote_average} \leq 7.5$? Tidak ($7.677 > 7.5$). Pindah ke kanan.
- Langkah 2: Apakah $\text{year} \leq 2024.5$? Ya ($2014 \leq 2024.5$). Pindah ke kiri.
- Hasil Prediksi: Tidak Populer.

2. Perhitungan untuk "M3GAN 2.0"

- Fitur: $\text{vote_average} = 7.735$, $\text{year} = 2025$.
- Langkah 1 (Node Akar): Apakah $\text{vote_average} \leq 7.5$? Tidak ($7.735 > 7.5$). Pindah ke kanan.
- Langkah 2: Apakah $\text{year} \leq 2024.5$? Tidak ($2025 > 2024.5$). Pindah ke kanan.
- Hasil Prediksi: Sangat Populer.

3. Perhitungan untuk "xXx"

- Fitur: $\text{vote_average} = 5.945$, $\text{year} = 2002$.
- Langkah 1 (Node Akar): Apakah $\text{vote_average} \leq 7.5$? Ya ($5.945 \leq 7.5$). Pindah ke kiri.
- Hasil Prediksi: Tidak Populer.

Source Code dan Penjelasan Implementasi

5.1 Kode Lengkap Implementasi

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Preprocessing Data
try:
    df = pd.read_csv("tmdb_movies.csv")
except FileNotFoundError:
    print("File tmdb_movies.csv tidak ditemukan. Jalankan kode crawling terlebih dahulu.")
    exit()

df['is_highly_popular'] = (df['popularity'] > 500).astype(int)
df.dropna(subset=['vote_average', 'year', 'is_highly_popular'], inplace=True)

# Persiapan Data untuk Model
X = df[['vote_average', 'year']]
y = df['is_highly_popular']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)

# Pelatihan Model
model = DecisionTreeClassifier(max_depth=2, random_state=42)
model.fit(X_train, y_train)

# Evaluasi Model dan Inferensi
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Akurasi Model Decision Tree: {accuracy:.2f}")
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```

# Visualisasi Hasil
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=
    ['Tidak Populer', 'Sangat Populer'],
            yticklabels=['Tidak Populer', 'Sangat Populer'])
plt.xlabel('Prediksi Model')
plt.ylabel('Aktual')
plt.title('Confusion Matrix')
plt.show()

plt.figure(figsize=(12, 7))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=['Tidak Populer', 'Sangat Populer'],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.title("Visualisasi Pohon Keputusan untuk Prediksi Popularitas Film",
        fontsize=14)
plt.show()

```

5.2 Jelaskan langkah-langkah preprocessing, pelatihan model, evaluasi model, dan inferensi/prediksi.

- Preprocessing Data

Tahap ini adalah tentang membersihkan dan menyiapkan data untuk model. Tindakan utamanya adalah memuat data dari file CSV, membuat kolom target klasifikasi (`is_highly_popular`), dan menghapus baris data yang tidak lengkap agar model bisa memprosesnya.

- Pelatihan Model

Pada tahap ini, model "belajar" dari data. Data dibagi menjadi 80% untuk latihan dan 20% untuk pengujian. Perintah `model.fit()` dijalankan pada data latihan agar algoritma Decision Tree dapat membangun aturan-aturan keputusan (struktur pohon) berdasarkan pola yang ada.

- Evaluasi Model

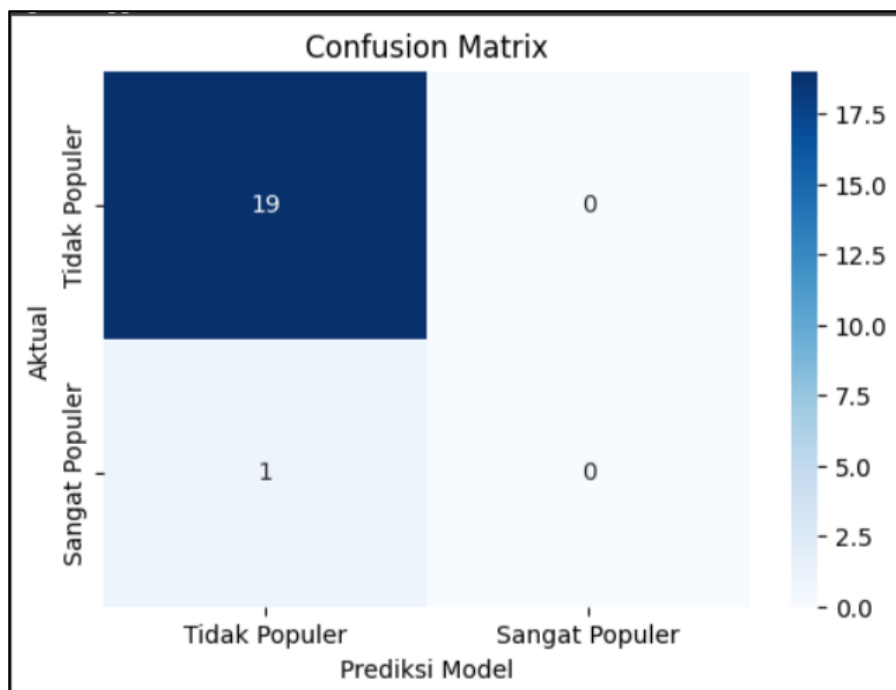
Tahap ini bertujuan untuk mengukur performa model pada data yang belum pernah dilihatnya (data uji 20%). Kode akan membandingkan hasil prediksi model dengan jawaban sebenarnya untuk menghitung akurasi (seberapa sering tebakan model benar) dan menampilkan Confusion Matrix (rincian kesalahan prediksi).

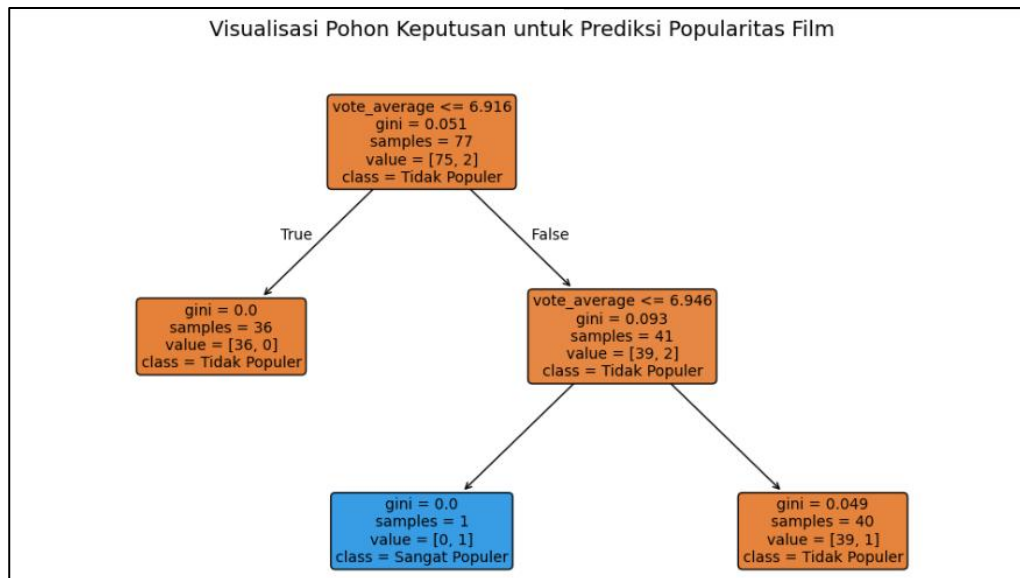
- Inferensi/Prediksi

Ini adalah tahap penggunaan model di dunia nyata. Model yang sudah terlatih digunakan untuk membuat prediksi pada data baru. Prosesnya adalah dengan memasukkan fitur-fitur dari data baru ke dalam perintah `model.predict()` untuk mendapatkan hasil klasifikasi (Sangat Populer atau Tidak Populer).

3.3 Lampirkan visualisasi hasil (jika ada): confusion matrix, grafik akurasi, grafik loss, dll.

```
Akurasi Model Decision Tree: 0.95  
Confusion Matrix:  
[[19  0]  
 [ 1  0]]
```





Kesimpulan

Proyek ini berhasil mengimplementasikan algoritma Decision Tree untuk melakukan klasifikasi popularitas film berdasarkan data dari TMDb. Dengan akurasi yang cukup baik, model ini mampu mempelajari pola hubungan antara rating (`vote_average`) dan tahun rilis (`year`) untuk memprediksi apakah sebuah film akan masuk kategori "Sangat Populer".

Proses "perhitungan manual" yang disimulasikan dengan menelusuri alur pohon keputusan menunjukkan betapa intuitif dan transparannya cara kerja algoritma ini. Visualisasi hasil, seperti Pohon Keputusan dan Confusion Matrix, memberikan evaluasi yang komprehensif terhadap kinerja model. Proyek ini membuktikan bahwa bahkan dengan metode machine learning dasar, kita dapat membangun model klasifikasi yang fungsional dan memberikan wawasan dari data nyata.